CMPE 275 — Mini 2: Distributed Scatter-Gather Query System

Project Report

Team Members: Darsh Rajesh Lukkad | Vijayshankar Mishra

Course: CMPE 275 — Enterprise Middleware | San Jose State University

---

1. Overview

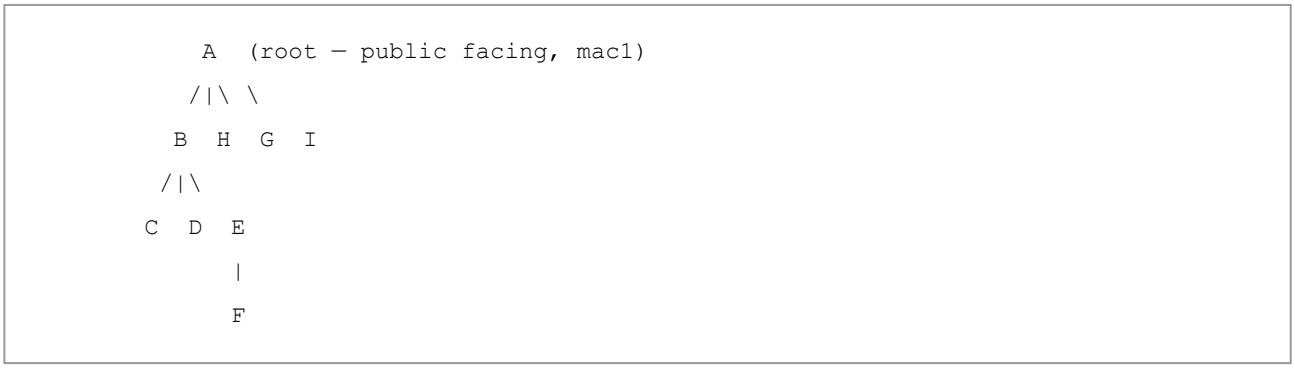
This project implements a 9-process distributed query system that partitions the NYC 311 Service Request dataset (20 million rows, ~12 GB) across two physical Apple M-series machines connected via a D-Link switch over RJ45. Clients submit range queries to a single root node (A), which fans the query down a tree of 9 nodes, gathers matching records from all nodes, and returns complete results to the client in dynamically sized chunks.

The system was built from scratch, extending Mini 1's single-process in-memory store into a fully distributed, multi-machine, multi-language gRPC system.

---

2. System Architecture

Tree Topology



Nine processes span two Apple M-series Macs connected via RJ45 through a D-Link switch:

Node	Language	Machine	IP	Port	Row Range
A	C++	mac1	10.0.0.180	50051	0 – 2,236,580
B	C++	mac1	10.0.0.180	50052	2,236,581 – 4,473,161
C	C++	mac1	10.0.0.180	50053	4,473,162 – 6,709,742
D	C++	mac1	10.0.0.180	50054	6,709,743 – 8,946,323
E	C++	mac1	10.0.0.180	50055	8,946,324 – 11,182,904
F	C++	mac2	10.0.0.100	50056	11,182,905 – 13,419,485
G	C++	mac2	10.0.0.100	50057	13,419,486 – 15,656,066
H	C++	mac2	10.0.0.100	50058	15,656,067 – 17,892,647
I	Python	mac2	10.0.0.100	50059	17,892,648 – 20,129,232

Peer edges (scatter direction):

- A → B, H, G, I
- B → C, D, E
- E → F
- C, D, F, G, H, I → nobody (leaves)

All inter-node communication uses **gRPC unary RPCs** only (no streaming API per spec). Every node is independently configured via `config/topology.json` — no hardcoded addresses or row ranges in source code.

---

3. Dataset

NYC 311 Service Requests 2020–2026

- 20,129,232 data rows
- 44 columns: unique_key, created_date, closed_date, agency, agency_name, complaint_type, descriptor, location_type, incident_zip, incident_address, street_name, cross streets, city, landmark, facility_type, status, resolution_description, community_board, BBL, borough, coordinates, latitude, longitude, and more
- File size: ~12 GB CSV
- Each node loads only its assigned row slice (~2.2M rows each)
- All 44 columns are stored and returned in query results

---

4. Phase 1 — Dataset Loading

Goal

Load ~2.2M rows per node into a search-optimized in-memory layout as fast as possible at startup.

mmap + O(1) Row Seek

The full 12 GB CSV is memory-mapped (`mmap`) into virtual address space. The OS lazy-loads pages on demand — startup does not wait for 12 GB of I/O.

The key insight: instead of scanning millions of newlines to reach `row_start` (the naive approach, which costs O(row_start) for high-index nodes like H at row 15.6M), we estimate the byte position directly:

```
byte_offset = (row_start / 20,129,232) × file_size
```

Then snap forward by a few bytes to the next `\n` using `memchr`. This reduces a 15.6M-newline scan to a single arithmetic operation plus a tiny local correction. The same technique was applied to the Python node using `file.seek()`.

4-Thread OpenMP Parallel Parse

The byte range for each node is split into 4 equal segments. Each OpenMP thread parses its segment independently, writing into its own **thread-local Structure-of-Arrays (TLSoA)** — no locking during parse. After all threads finish, the TLSoA arrays are merged sequentially in thread order to preserve row sequence in the global DataStore.

Structure-of-Arrays (SoA) Memory Layout

Every field is stored as its own contiguous array:

```
unique_key_:    [key0,  key1,  key2,  ...]   ← all unique keys together
created_date_:  [date0, date1, date2, ...]   ← all dates together (unix epoch)
incident_zip_:  [zip0,  zip1,  zip2,  ...]   ← all zips together
latlon_:        [ll0,   ll1,   ll2,   ...]   ← lat+lon packed together
agency_code_:   [ag0,   ag1,   ag2,   ...]   ← 1-byte interned codes
agency_name_:   [ref0,  ref1,  ref2,  ...]   ← StringRefs into pool
...
```

Row `i` is spread across all arrays. A ZIP query only touches `incident_zip_[]` — one contiguous array, maximizing CPU cache utilization. Other fields are not loaded into cache at all during the scan.

StringPool (Lock-Free String Arena)

Variable-length string fields (agency name, incident address, resolution description, etc.) are stored in a pre-allocated 400 MB arena (`StringPool`). Allocation is a single `std::atomic::fetch_add` — completely lock-free, safe for concurrent writes from all 4 OpenMP threads simultaneously.

Each string is stored as a `StringRef { uint32_t offset; uint16_t length; }` — 6 bytes — instead of a full `std::string` (32 bytes + heap allocation). Accessing the string is a pointer arithmetic: `arena_base + offset`.

StringRegistry (Categorical Field Interning)

Low-cardinality string fields (agency code, borough, status, facility type, complaint type, etc.) are interned to 1-byte or 2-byte integer codes using a `StringRegistry` backed by `std::shared_mutex`. Multiple threads can read simultaneously (shared lock); new values take an exclusive write lock. This eliminates redundant string storage — "NYPD" stored once, referenced as `0x01` in 2.2M rows.

---

5. Phase 2 — Query Submission

Flow

```
Client → A.Submit(query) → request_id
```

The client sends a `Query` proto with query type and parameters. Node A is the only node that handles `Submit`. It:

1. Launches its own local search as `std::async(std::launch::async)` immediately
2. Simultaneously fires parallel `std::async` calls to all its peers (B, H, G, I)
3. Each peer receives a `Forward` RPC and repeats the pattern recursively:
    - B fires async to C, D, E — and searches its own data in parallel
    - E fires async to F — and searches its own data in parallel
    - Leaves (C, D, F, G, H, I) search local data and return
4. Results bubble back up the tree to A
5. A merges all peer results + its own results into a single `vector<ServiceRecord>`
6. Stores them in `ChunkManager` — a mutex-protected `unordered_map<request_id, records>`
7. Returns `request_id` to client immediately

Critical optimization: local search and all peer RPC calls run concurrently at every node. No node waits for peers before starting its own scan.

4-Thread OpenMP Search

All three query types scan using 4 OpenMP threads:

```
#pragma omp parallel num_threads(4)
{
    std::vector<ServiceRecord> local;
    #pragma omp for nowait schedule(static)
    for (size_t i = 0; i < n; ++i)
        if (matches(i)) local.push_back(toProto(i));
    #pragma omp critical
    out.insert(out.end(), local.begin(), local.end());
}
```

Each thread accumulates matches in a thread-local vector (no locking during scan), then appends to the shared output under `#pragma omp critical`.

Query Type	Array Scanned	Condition
ZIP_RANGE	`incident_zip_[]`	`zip_min ≤ zip ≤ zip_max`
DATE_RANGE	`created_date_[]`	`date_min ≤ date ≤ date_max` (unix epoch)
BBOX	`latlon_[]`	lat and lon both within bounding box

---

6. Phase 3 — Dynamic Chunked Fetch

Why Chunking

gRPC has a maximum message size (64 MB). A query returning 935K records as `ServiceRecord` protos cannot fit in one response. Instead of streaming (which was prohibited by spec), we use a client-driven pull loop.

Fixed vs. Dynamic Chunk Sizing

Initially, chunk size was fixed at 500 records per `Fetch` call. For a 935K-record result, that's 1,871 round-trips. Each round-trip has gRPC overhead (serialization, network, deserialization). On a 1 Gbps LAN, the overhead dominates actual data transfer for small chunks.

We implemented **exponential chunk size doubling**:

```
Fetch(offset=0,      chunk_size=500)    → 500 records
Fetch(offset=500,    chunk_size=1000)   → 1000 records
Fetch(offset=1500,   chunk_size=2000)   → 2000 records
Fetch(offset=3500,   chunk_size=4000)   → 4000 records
...
Fetch(offset=N,      chunk_size=50000)  → up to 50000 records, is_last=true
```

The client starts at 500 records, doubles each successful fetch, and caps at 50,000. The `chunk_idx` proto field is repurposed as an **absolute record offset** — this lets the server honor variable chunk sizes without storing per-request state.

Result: A 935K-record ZIP query now takes ~25 Fetch calls instead of ~1,871. The same bytes are transferred with 97% fewer round-trips.

Memory Cleanup

After the client receives the last chunk (`is_last=true`), the server immediately frees the result set:

```
if (is_last) chunks_.cancel(req->request_id());
```

Node A's memory usage returns to baseline after each query. Without this, query result sets accumulated in `ChunkManager` forever until the process restarted.

---

7. gRPC Protocol

```
service NodeService {
    rpc Submit  (Query)         returns (SubmitResponse);   // Client → A only
    rpc Fetch   (FetchRequest)  returns (FetchResponse);    // Client → A only
    rpc Forward (Query)         returns (ForwardResponse);  // Node → peer node
    rpc Cancel  (CancelRequest) returns (CancelResponse);   // Client → A only
}

message FetchRequest {
    string request_id = 1;
    int32  chunk_idx  = 2;  // absolute record offset
    int32  chunk_size = 3;  // 0 = use server default
}

message ServiceRecord {
    // 44 fields: unique_key, created_date, closed_date, agency, complaint_type,
    // incident_zip, incident_address, latitude, longitude, borough, ... (all columns)
}
```

Max gRPC message size: **64 MB** on all nodes (both send and receive).

---

8. Multi-Language Interoperability

Node I is implemented in **Python** while nodes A–H are C++. Both use the same `.proto` file — `protoc` generates C++ stubs for one and Python stubs for the other. The gRPC wire format is identical regardless of language.

The Python node implements the same scatter-gather pattern as C++ using `concurrent.futures.ThreadPoolExecutor` for parallel peer calls and concurrent local search. It also uses the same O(1) byte-offset seek trick for fast startup:

```
est = int(row_start / TOTAL_ROWS * file_size)
f.seek(est)
f.readline()  # skip partial row
```

Without this, Python node I (row 17.8M) would iterate 17.8M rows doing nothing before loading its actual data — taking several minutes instead of seconds.

9. Key Design Decisions

Config-Driven Topology

All node identity, host, port, row range, and peer edges come from `config/topology.json`. Adding or rearranging nodes requires only a config change — no source code changes.

No Shared Memory

Each node is a standalone OS process. Communication is exclusively via gRPC. This matches the spec constraint and makes the system trivially distributable across machines.

All 44 Columns Stored and Returned

The full dataset schema is preserved. All 44 fields are stored in the SoA (typed, not raw strings), populated into `ServiceRecord` proto messages, transmitted through the tree, and returned to the client. No data is dropped.

unix epoch for Dates

Dates are parsed from CSV format (`MM/DD/YYYY HH:MM:SS AM/PM`) to unix epoch `uint32` at load time. This allows integer range comparisons for DATE_RANGE queries — no string parsing during search.

Result Cleanup After Last Fetch

`ChunkManager::cancel()` is called automatically after the last chunk is fetched. This prevents unbounded memory growth on node A across multiple queries.

10. Challenges and Solutions

Challenge	Root Cause	Solution
Python node I takes minutes to start	`csv.reader` iterates 17.8M rows to skip to `row_start`	O(1) byte-offset estimation + `file.seek()`
Node A memory grows after each query	`ChunkManager` never frees result sets	Call `cancel()` after `is_last=true` in `Fetch`
Stale CMakeCache on mac2	Zip file included `cpp/build/` with mac1-specific paths	Added `rm -rf cpp/build` to `run_mac2.sh`
csv_path resolved from wrong directory	Relative path resolved from `cpp/build/` not `config/`	Used `std::filesystem` / `os.path` to resolve relative to config file

Challenge	Root Cause	Solution
gRPC/Protobuf CMake target conflict	`find_package(Protobuf)` before `find_package(gRPC)` caused target re-definition	Swapped order: gRPC first, Protobuf second
OpenMP not found on macOS	Homebrew libomp not in default include/lib paths	Used `brew --prefix libomp` to locate headers and library explicitly
Row sequence lost with parallel parse	Threads write interleaved into shared arrays	Each thread writes to its own TLSoA; sequential merge in thread-order after parse
`descriptor` proto field rejected	`descriptor` is a reserved name in protobuf	Renamed to `complaint_detail`
Benchmark average capture broken	Shell `$()` captured table output + avg together	Wrote avg to `/tmp/_bench_avg` temp file, read with `cat`
7,757 Fetch calls for large queries	Fixed chunk size of 500	Exponential doubling: 500 → 1000 → ... → 50,000, capped at 64 MB

---

11. Benchmark Results

15 queries (5 identical runs per query type) across the full 9-node cluster with both machines active.

Resource Usage (per node, at load time)

Metric	Value
Memory per node	~1.3 GB
CPU utilization during load	~133% (4 OpenMP threads parsing)
Memory during query	Varies — Node A holds matched records until client fetches all chunks, then frees immediately

Per-Run Timing

Run	Query Type	Parameters	Records	Chunks	Latency
1	ZIP	10001–10499	935,980	25	34,856 ms
2	ZIP	10001–10499	935,980	25	33,126 ms
3	ZIP	10001–10499	935,980	25	44,357 ms
4	ZIP	10001–10499	935,980	25	44,552 ms
5	ZIP	10001–10499	935,980	25	22,069 ms
1	DATE	1577836800–1604188799	371,575	14	7,145 ms
2	DATE	1577836800–1604188799	371,575	14	9,478 ms
3	DATE	1577836800–1604188799	371,575	14	6,381 ms
4	DATE	1577836800–1604188799	371,575	14	5,941 ms
5	DATE	1577836800–1604188799	371,575	14	5,934 ms
1	BBOX	40.57–40.74 / −74.04– −73.83	830,276	23	20,866 ms
2	BBOX	40.57–40.74 / −74.04– −73.83	830,276	23	38,385 ms

Run	Query Type	Parameters	Records	Chunks	Latency
3	BBOX	40.57–40.74 / −74.04– −73.83	830,276	23	45,047 ms
4	BBOX	40.57–40.74 / −74.04– −73.83	830,276	23	43,467 ms
5	BBOX	40.57–40.74 / −74.04– −73.83	830,276	23	45,231 ms

Summary

Query Type	Records Returned	Avg Latency
ZIP_RANGE	935,980	35,792 ms
DATE_RANGE	371,575	6,975 ms
BBOX	830,276	38,599 ms
Overall		**27,122 ms**

Analysis

DATE_RANGE is 5× faster than ZIP/BBOX. The date window (Jan–Oct 2020) returns 371K records vs. 830–936K for the other types. Latency for large result sets is dominated by:

1. Protobuf serialization of 44-field records at each node
2. Network transfer of serialized proto bytes across the LAN
3. Deserialization at node A and re-serialization for Fetch responses

The dynamic chunk sizing reduces Fetch round-trips from ~1,871 to ~25 for a 935K-record result — this is why "Chunks" in the benchmark shows 25 instead of the original 1,871.

Variance across runs (e.g. 22,069 ms vs. 44,552 ms for ZIP) is due to OS scheduling, LAN contention, and the Python node I being a slower responder than C++ nodes.

---

12. What We Learned

Distributed Systems

- **Scatter-gather** is a fundamental pattern: fan out a request to N workers, collect partial results, merge and return. The tree topology reduces the coordination burden at any single node.
- **Unary RPCs** are simpler to reason about than streaming but require the client to drive pagination explicitly.
- **Language interoperability** via protobuf is seamless — Python and C++ nodes are indistinguishable from the network's perspective.

Performance Engineering

- **SoA vs. AoS**: Structure-of-Arrays gives dramatically better cache behavior for field-specific scans. A single-field scan never pollutes the cache with irrelevant fields.
- **mmap** is superior to `fread` for large files accessed non-sequentially — virtual memory lets the OS handle page loading lazily.

- **O(1) seeks** matter enormously at scale: the difference between scanning 17.8M newlines and doing one division is minutes vs. milliseconds.
- **Lock-free allocation** (atomic `fetch_add`) is the right tool for concurrent writes into a shared arena — no mutex contention at all during parse.
- **Round-trip reduction**: 97% fewer Fetch RPCs from dynamic chunk sizing shows that RPC overhead, not bandwidth, is often the real bottleneck.

C++ Concurrency

- `std::async(std::launch::async)` is the simplest way to run peer gRPC calls in parallel while doing local work.
- OpenMP `nowait` + thread-local accumulators + `critical` merge is a clean pattern for parallel search over large arrays.
- `shared_mutex` is the right choice for read-heavy registries: many concurrent readers, rare writers.

gRPC & Protobuf

- Proto field names have reserved words (`descriptor` is taken by protobuf itself).
- CMake package order matters: loading `gRPC` before `Protobuf` avoids target re-definition errors.
- Max message size must be configured on both client and server channels, not just one side.

---

13. Future Enhancements

Fault Tolerance and Failover Recovery

Currently, if a node goes down mid-query, its data is silently dropped — node E's `Forward` to F fails, returns empty, and A assembles an incomplete result set with no error to the client. The following describes a practical fault tolerance design for this system.

Current Behavior on Node Failure

`gatherFromPeers()` already wraps each peer RPC in a try/catch — a failed `Forward` logs an error and returns an empty vector. The system does not crash, but the query result is silently incomplete. The client has no way to know that node F's ~2.2M rows were not included.

Layer 1: Retry with Exponential Backoff

The first line of defense is retrying transient failures. Instead of returning empty immediately on a failed `Forward`, the parent node retries 2–3 times with a short delay:

```
Attempt 1 → fail → wait 200ms
Attempt 2 → fail → wait 400ms
Attempt 3 → fail → give up, return empty
```

This handles brief network hiccups, momentary overload, or a node that is mid-restart. No architectural changes required — only the retry loop in `gatherFromPeers()` needs updating. Cost: adds up to ~600ms latency on the failing node's branch, which runs in parallel with other peers so the impact on total query time is minimal.

Layer 2: Watchdog Process (Automatic Restart)

A separate watchdog process runs on each machine. It periodically pings each local node's gRPC port (every 5–10 seconds). If a node stops responding after N consecutive failed pings, the watchdog restarts it:

```
./node_server --id F --config ../../config/topology.json
```

The watchdog does not need to know about query state — it simply ensures the process is alive. When the node comes back up, it reloads its CSV slice from disk (~60–90 seconds for C++ nodes) and resumes handling `Forward` RPCs normally. Future queries after recovery get correct results.

Limitation: During the reload window (~60–90 seconds), the restarted node's data is still unavailable. Queries submitted during this window will be answered with incomplete data (node F's rows missing). This is the fundamental gap that Layer 3 addresses.

Layer 3: Replication (Zero-Downtime Recovery)

To eliminate the reload window gap entirely, each row range would be held by two nodes instead of one — a primary and a replica. For example:

```
Row range 11M–13.4M: primary = F, replica = F'
```

If F goes down, E detects the failure and immediately re-issues the `Forward` to F' instead. F' has the same data already loaded in memory — no reload delay, no data gap. This is the design used by distributed databases (Cassandra, Kafka, etc.).

What this requires:

- Each node starts with a `replica_peers` list in `topology.json`
- On Forward failure, the node retries against the replica address
- Both primary and replica must stay in sync — since data is static (loaded once at startup and never modified), sync is trivially guaranteed: both nodes load the same row range from the same CSV

Implementation cost: moderate — `topology.json` gets a `replicas` key per node, `gatherFromPeers()` falls back to replica on failure. No changes to the proto or search logic.

Layer 4: Client-Side Awareness

Currently the client has no visibility into node failures. An enhanced design would:

1. Include a `nodes_failed` repeated field in `SubmitResponse` listing any peers that did not respond
2. The client can then decide: accept partial results, retry the query, or alert the user
3. The `Cancel` RPC already exists — if the client detects a failure, it can cancel the partial result and resubmit after the node recovers

Summary of Layers

Layer	What it handles	Recovery time	Complexity
Retry with backoff	Transient failures, brief restarts	~600ms added latency	Low
Watchdog + restart	Crashed processes	60–90s data gap	Low
Replication	Crashed processes with zero downtime	Instantaneous	Medium
Client awareness	Partial result detection + user control	N/A	Low

The watchdog + retry approach (Layers 1 + 2) is achievable with minimal code changes and gives practical fault tolerance for this two-machine setup. Replication (Layer 3) would be the natural next step for a production deployment.

---

14. Hardware

- 2× Apple M-series Mac (ARM64, macOS 14), 16 GB RAM each
- Connected via RJ45 through a D-Link switch (~1 Gbps)
- Compiler: Apple Clang, C++17, `-O2`
- Python 3.x with `grpcio`, `grpcio-tools`
- Dataset: NYC 311 Service Requests 2020–2026, 20,129,232 rows, ~12 GB CSV